Handoff: Babbage's Hall

For a new chat: how to work, build, test, and what's next · 14 November 2026

Read this first in a new chat. It says how William likes to work, where everything is, how to build and test, what has been learned, and what to do next. The list of machines is `MACHINES.md`; the hall itself is described in `README.md`. This file is updated whenever a chat ends.

1. What this is

- **Babbage's Hall** (`demo/WyomingWill/BabbagesHall/`), a Victorian engineering hall in the Bullet Physics Playground (bpp, **github.com/bullet-physics-playground/bpp**; William pushes to the branch **`test-harness`** (since 2026-11-11), which Jakob merges into `master`): cast iron, brass, a glass roof, holding **machines that compute**. A separate museum from William's Euler's Gallery (`../EulersGallery/`), built on its machinery (separate worlds, sandboxes, walking with Tab).
- **Every machine can be 3D printed and built.** Each exhibit's parts are OpenSCAD sources (the single source for the simulation and the prints), a `make-stls.sh` makes the STL files, and its README lists what to buy and how to assemble it. What is printed is what is simulated: no shortcuts in the parts that compute, and anything idealised is said on the placard and in FINDINGS.
- **The walk asks more of you as you go:** from what asks least (one idea, little mechanism) to what asks most, Babbage's Difference Engine at the far end. `MACHINES.md` has the order and why (re-ordered 2026-11-14: it was by interest alone).
- Built so far, in walking order: **the Domino Half Adder** (west 1), **the Differential** (east 1), **the Pascaline** (west 2), **the Galton board** (west 3, since 2026-11-12), **the splitter board** (east 3, facing it, since 2026-11-14), **Lehmer's number sieve** (west 4), and the **Digi-Comp II** (east 4: it counts, adds, multiplies, subtracts and divides, runs programs, and in AUTO runs a multiply by itself, or a division to OVERFLOW's halt; its weight and fly are parts: a cup of balls and a bought rotary damper; each program has only its parts on the board, and the operator's hand is shown). Covered bays wait for Amsler's planimeter (west 2), Kelvin's tide predictor (east 4), the Differential Analyser (west 5) and the Difference Engine (east 5): ten bays, five rows.
- William M. Spears (`williammspears@gmail.com`) owns the WyomingWill demos; Jakob (koppi) wrote bpp. Clocks are William's own separate collection.

2. How William likes to work

- **Deliver every change as a patch plus the git commands.** He applies patches in a checkout called `bpp-push`, pushes, then pulls in his installation checkout (which has `pull.ff only`; never apply patches there). Give the commands as below, and say if bpp must be rebuilt (only when C++ changed).

```
git pull
git am ~/Downloads/<name>.patch
git push
```

- **Every patch includes all its documents, PDFs too.** Every `.md` has a typeset `.pdf` beside it (`./make-pdfs.sh`, or `tools/md2pdf.py` for one); rebuild the PDF whenever its `.md` changes. Euler's

Gallery and the Rec Room have README.pdfs made with `../ArtificialPhysics/tools/md2pdf.py`.

- **Build patches on `origin/test-harness`**, where he pushes (a clone fetches only `master`: `git fetch origin test-harness:refs/remotes/origin/test-harness`). To check something landed, look there, not on `master`.
- **Before sending a patch:** check it applies to the current `origin/test-harness` (fetch first; a patch built on one he hasn't applied fails, as the first `MACHINES.md` patch did); no trailing spaces (`git diff --check`); `tools/check-all.sh` passes; the visuals looked at (`tools/shots.sh`). A re-sent patch gets a new file name, so a stale download can't be applied.
- Commits are authored `William M. Spears <williammspears@gmail.com>`, with the Co-Authored-By trailer for Claude. Stage only the files you changed.
- He looks at the exhibits in the GUI and reports what he sees with screenshots: take his reports seriously and reproduce them before answering, and measure when the pictures don't settle it.
- Plain explanations; measure before claiming; say what was and wasn't tested.
- **His bpp is built with Lua 5.2;** a cloud workspace's is usually 5.1. Write Lua that runs on both: no bare `setfenv`/`getfenv` (give `loadfile` its environment with `"t", _ENV` on 5.2, as the exhibits' table loaders do), no `goto`. `tools/check-lua52.sh` checks the one place this has bitten (the table loader) under a standalone `lua5.2`.

3. Where everything is

Path	What
`hall.lua`	The hall: architecture, the EXHIBITS list (walking order, placards, `floorY`), sandboxes, walking and camera flights, resting, the cost meter
`exhibit-table.lua`	The standard exhibit table (README "Exhibit standard")
`MACHINES.md`	All 48 machines considered, in walking order (five tiers), scored for interest and complexity, with how each would go in bpp and be printed; the rope test
`README.md`	The hall: bays, keys, how it works, the cost meter, the exhibit standard, notes for exhibit authors
`HANDOFF.md`	This file
`make-pdfs.sh`, `tools/md2pdf.py`	The PDFs
`make-labels.py`, `make-lecterns.py`, `hall-meshes/`	The placards' lettering (Pillow, NumPy, DejaVu fonts); the lecterns' cards (PNG)
`tools/check-all.sh`	Every exhibit's tour, on its own and in the hall
`tools/shots.sh`, `tools/keys.sh`	Screenshots on a virtual screen; with keys, hover, camera flights
`tools/check-lua52.sh`	The Lua 5.2 sandbox check
`tools/rope-test.lua`	The rope experiment (for the tide predictor)
`exhibits/DigiComp2/tests/toggle.lua`, `tests/switch.lua`, `tests/junction.lua`	One Digi-Comp flip-flop alone (with its foot, done hole and clear lane); one switch; the piece of board where the memory meets the accumulator
`exhibits/DigiComp2/MANUAL.md`, `manual/`	The Digi-Comp's illustrated manual; `manual/make-figures.sh` redraws its figures from the geometry (lua, python3, matplotlib). `tools/md2pdf.py` puts a `.md` file's images into its PDF
`exhibits/<Name>/`	One folder per exhibit: its script, `parts/` (OpenSCAD), `make-stls.sh`, `README.md`/`.pdf`, `FINDINGS.md`/`.pdf`

Related: `../EulersGallery/` (the gallery the hall's machinery came from; its tables still follow its own sizes, see §7), `../RecRoom/` (games room: the cost meter and suspending came from there), `demo/claude/digicomp.lua` (an incomplete Digi-Comp II, a starting point), `module/scad/gearsv50.lua` and the Differential's `parts/gearsv50.scad` (bevel gears).

4. Building, running, testing

```
git clone https://github.com/bullet-physics-playground/bpp ~/bpp && cd ~/bpp
apt-get update
apt-get install -y libbullet-dev libluabind-dev liblua5.1-0-dev liblua5.2-dev \
  libqglviewer-dev-qt5 libsdl2-mixer-dev libassimp-dev qtbase5-dev libglew-dev \
  xvfb xdotool imagemagick openscad lua5.2 povray
pip install --break-system-packages trimesh manifold3d markdown playwright
mkdir -p build && cd build && qmake ../bpp.pro CONFIG+=release && make -j2 release
```

That gives `build/release/bpp` (a few minutes to ten). The tools find it there, in `release/`, or on the path; or set `BPP`.

- **Headless:** from the top folder, `build/release/bpp -f demo/WyomingWill/BabbagesHall/hall.lua -n 600`. `HALL_START=pascaline` starts at an exhibit. The exhibits run on their own the same way, each with a tour that prints `RESULT <name> tour: PASS`.
- **GUI on a virtual screen:** `tools/shots.sh` (see its header); `LIBGL_ALWAYS_SOFTWARE=1`, so drawing is slow and the simulation runs slower than real time there. In the GUI, `print` goes to bpp's console pane, not the terminal: write to a file (the meter's `HALL_METER_FILE`, the flights' `HALL_FLY_LOG=file`) to read it.
- **Printed parts:** each exhibit's `make-stls.sh` (OpenSCAD 2021.01 here; William's is newer: avoid `assign()` and `polyhedron(triangles=)`). Check every STL is one watertight solid (trimesh), and that parts don't overlap at rest (manifold3d); OpenSCAD renders pictures under `xvfb-run`.
- **POV-Ray:** `build/release/bpp -f ... -n 2 -e` writes `export/<script>/` (untracked; delete it after); render with `povray +I<script>.pov +K2 +W800 +H450 +L<bpp folder>`.

5. What has been learned (bpp and Bullet)

- **OpenSCAD parts** are cached by the text asked for, not by the files' contents: put a checksum of every file the part includes into the text (each exhibit's `checksum()`). Set `collides` before `v:add`, the transform after. A body's origin must be its centre of mass.
- **`polyhedron()` faces go clockwise seen from outside.** OpenSCAD 2021's CGAL forgave the other way when it joined pieces; William's newer OpenSCAD (Manifold) doesn't, and such a part comes out inside out: drawn with dark slivers when bpp's shadows are off (the shader hides it), and a bad STL. Check a new polyhedron's signed volume (trimesh, or manifold3d on its points). Fixed 2026-11-04 in the Differential's gears and the sieve's cam ring.
- **Two bpp processes building the same new part at once** can collide in the mesh cache: run tests one at a time after a part changes.
- **Gears:** mesh-on-mesh (GImpact) bevel teeth jam; a compound of convex hulls (one per tooth plus the body, from the same SCAD source) meshes well (`exhibits/Differential/FINDINGS.md`).
- **Hinge angles** (`getHingeAngle`) are clockwise-positive as seen down the hinge's axis, the opposite of a right-handed turn. A hinge's motor speed counts the same way as its angle (measured on the Digi-Comp's wheel).

- **A pin's friction** can be had from a hinge's motor held at no speed with a small impulse a step (`enableAngularMotor(true, 0, torque * step)`); the Digi-Comp's rockers use it, from PLA on PLA at the boss.
- **Over-centre parts** (a rocker whose centre of mass is above its pin) rest in a very shallow dip: with no friction they bounced back out of it, with pin friction they stop wherever they're left. Give them weight high up (the Digi-Comp's rockers have a steel ball pressed in) and test both extremes.
- **Walls as boxes** run past their ends by half their thickness if made `length + thickness` long, so the joints are solid: check where such an end meets a moving part (the Digi-Comp's feeder wheel rubbed two), or narrows a channel (two jammed the Digi-Comp's clear lane). Mitre a lane's walls at its bends (`laneWalls` in the Digi-Comp's geometry).
- **A ball pushing a part out of its way** pushes it only until it can squeeze past, which may be nowhere near where the part should end up: to push it all the way, guide the ball (the Digi-Comp's clear lane narrows beside each foot).
- **Two ways into one place:** trace the ball that comes in first (its position every frame) before giving the other one a way in; a ball rolling down a wall must pass under the ends of the walls above it with room for the ball (the Digi-Comp's joining funnels, `FINDINGS.md`). Lanes on one level can't cross: their order is fixed by where they start and end, so plan a board in a drawing of every wall first.
- **A part with holes** (the Digi-Comp's upper plate) is a static body of boxes round the holes, drawn not by them (`transparency = 1`) but by the OpenSCAD part. `transparency` near 0.35 draws a see-through plate well.
- **Luabind objects** from `v:eachContact` can't be compared with `==`; look them up as table keys (`names[obj]`).
- **Small steps:** the exhibits take 2400 steps a simulated second, stepped by hand in `preSim` (`common.setTiming(STEP, 0, STEP)`); the teeth and the dominoes need it. Sleeping is off for moving parts (state 4), so each exhibit **rests** (stops stepping) once nothing moves; it changes nothing computed.
- **Springs** that matter are simulated as the printed part: the Pascaline's detent is a body on a hinge turned back by the leaf's stiffness from PLA (3EI/L), and its working range of thickness was found by sweeping it.
- **A part driven by a weight through a damper** moves slowly wherever only a small force drives it: the Digi-Comp's escapement, whose pin must creep off the anchor's second finger, took up to 1.8 s to let go with the damper where the torque-and-cap had taken half a second. Measure such waits with the real drag, at both ends of its tolerance.
- **An object's `pos` (and `trans`) is its drawing's**, which follows its body only after a step: read a body that has just been placed from the body (`getCenterOfMassPosition()`).
- **`getCenterOfMassTransform()` is the body's own transform**, not a copy (and `getCenterOfMassPosition()` its own vector): keep a pose by copying it (`btTransform(tr:getRotation(), btVector3(p.x, p.y, p.z))`). Kept as is, it moves with the body (the Digi-Comp's parts were "put back" where they had been sent). An object's `.trans` is a copy.
- **Taking parts away finds hidden jobs.** A wall drawn as one part's may do another's work (the Digi-Comp's clear lane steered done balls; a done lane's wall was its hole's backstop). Test each reduced machine, in AUTO too: in MANUAL a ball stopped in the wrong place often still counts as done.
- **Impulses applied in `preSim` before a step** (a cord's pull, a drag) change a held body's velocity for an instant, until the step's contacts take it back: read speeds for "is it moving?" before applying them (the Digi-Comp's hand thought the held wheel was turning, and never began).
- **Rope:** none built in; a rope of beads on ball joints works if the rope isn't far lighter than what hangs on it (`MACHINES.md`, "Rope").

- **Mouse:** scripts get `onHover` (after half a second still, no button down) but no clicks; keys override bpp's own one-letter keys.
- `btTransform` has no `getOrigin` in bpp's Lua; `.trans`, `:mult` and `getCenterOfMassTransform` are what there is.
- **Compounds:** a `btCompoundShape` in bpp's Lua takes only a few dozen children (each costs one of luabind's few dependency slots; past them bpp stops with an assertion in `object_rep.cpp`). Build big bodies as trees of small compounds, neighbours together (Lehmer's sieve's `tree()`), and don't add and remove children over and over: move them (`updateChildTransform`), as the sieve's pegs do.
- **Cylinders:** bpp's Lua has `btCylinderShapeX` and `btCylinderShapeZ` but no constructor for the upright `btCylinderShape`: stand a Z one up.
- **Convex pieces** from a 2D outline: `exhibits/LehmerSieve/parts/make-geometry.py` (shapely's constrained triangulation, merged while convex); check that the pieces' areas add up to the outline's. Corners inside an outline cost pieces; draw holes and inner arcs coarser for the contacts than for the print.
- **An OpenSCAD part made a body** must be set to collide before `v:add` (the exhibits' `part()` turns collisions off).
- `v:eachContact(fn)` calls `fn(a, b, x, y, z, nx, ny, nz, distance, impulse)` for every contact: the quickest way to find what jams.
- **Geneva drives** index well but their locking disc only bounds a star's turn (about a degree either way): a sprung detent per star puts it on its place, and the disc's relief must clear the star's lead and lag (`exhibits/LehmerSieve/FINDINGS.md`).
- `.gitignore` at the top ignores `*.scad`: the hall's own `.gitignore` has `!*.scad`, and STL files are not kept (each `make-stls.sh` makes them).
- **Balls in two dimensions jam** (the Galton board, `FINDINGS.md`): a hopper of balls between two plates, emptying through a neck, clogged at three balls wide and at five; a ball perched on a pin under the neck held up an arch or a zigzag column; taps (rocking the board) did not clear it. A neck wide enough flooded the pins, and the crowded balls spread far too wide. A slotted wheel metering one ball at a time is what worked.
- **A mechanism hands every ball on the same way,** and the result leans its way: the Galton board's one metering wheel made the first pin a switch (moving a chute 1.5 mm flipped the lean); a funnel and neck hid it, until the bounce was changed and the lean changed sides. William asked whether the setup was asymmetric: turning the wheel the other way swapped every lean. Two mirror-image wheels, geared, fixed it (the mean within about 0.3 of 6 from a bounce of 0.3 to 0.55). If a result depends on a parameter that should only change its spread, look for an asymmetry; test by mirroring (`GB_LOG=3` shows how the balls leave the neck).
- **An invisible change is a different run.** Rewriting the Galton board's OpenSCAD sizes file (three empty lists added, nothing drawn differently) changed its tour's numbers (5.88/2.12 to 5.73/2.18); so does running it with an empty parts cache. The parts don't collide, but something about building them changes the order the solver meets the balls, and the board is chaotic. Neither run is wrong. Compare a change against the same cache, and keep a file's text unchanged when its meaning is.
- **Fair is not independent** (the Galton board, `FINDINGS.md` section 6): each pin split the balls 50/50, yet the bins came out wider than the binomial. William guessed why: a ball leaving a pin carries a bias to the next. Recording each ball's path (`GB_PATHS`, `paths.py`) measured it: the same way again two times in three, the first pair 98%. A deader ball remembered more (damping would have been the wrong fix), and taller rows forgot but let balls jump pins; no setting tried beat the equilateral lattice. When a count is wrong but every single step looks fair, measure the correlations between steps.

- **A full container looks like a jam:** balls left at the slide's level were sitting on a heaped, full drawer, and knocking did nothing. Size a container for a heap, not for flat packing.
- **Checked against bpp's new Params and ScriptKeys (2026-11-11):** `tools/check-all.sh` all passed on Jakob's rewrite, and in the window Tab, the panes and the sliders work.
- **Done (2026-11-12, revised 2026-11-13): the Galton board** (now west 3), in a new row of bays: the hall has ten now, the planimeter's covered bay (west 2) before it (by rank, 14 and 13), and everything after it one row on, on the same side (Pascaline west 3, Domino Half Adder east 3, Digi-Comp II west 4, tide predictor east 4, Differential Analyser west 5, Difference Engine east 5; the READMEs' bay numbers and the PDF subtitles with them). 150 steel balls (6 mm), 4 mm pins 12 mm apart, twelve rows from the first pin, thirteen bins with the binomial's heights for 150 engraved in gold; a hopper, **two mirror-image metering wheels** geared together behind the board and turned by one crank (the hand, a motor; the gears, a lock holding both to the crank's turn), a funnel under both and a straight neck onto the first pin; a slide under the bins, a drawer, the hand tipping the drawer back into the hopper (the pouring moves the balls, as the Digi-Comp's hand does), and knocking the board if balls stay wedged (idealised; not needed since the drawer was made deeper). A sign (the exhibit's) shows the run, the balls in the bins, the mean bin and the spread against 6.00 and 1.73. The tour: a run, empty and pour, a run (about 4 min headless; check-all allows 6000 frames, 6600 in the hall). The first patch (2026-11-12) had one wheel, and the board leant whichever way the bounce tipped it: William spotted the asymmetry; the second patch replaces the wheel with the pair. Over six runs at the usual settings the mean bin was 5.97 and the spread 1.94 (fair coins: 6.00, 1.73); fair from a bounce of 0.3 to 0.55 and at friction 0.15 and 0.45. At a bounce of 0.7 the spread passes the tour's 2.6. The bounce (0.4, ball on board) is a guess (no figure for steel on printed PLA was found); FINDINGS says how to measure it. Keep the crank to 0.75 turns a second: faster, two balls meet over the neck and jam. Why wider than fair coins (2026-11-13, a third patch): each pin is fair but remembers the last (the same way again two times in three; FINDINGS section 6, with the bounce, friction and row-spacing sweeps); the board was left as it is, the instrument kept (`GB_PATHS`, `paths.py`, and `GB_GEO="V=..."` for the rows). Left to try: feeding the first pin a little off-centre, since the first pair goes together 98% of the time. Splitters (2026-11-13, a fourth patch): a neck above every pin, tested on its own (`splitter-test.lua`, `SPLITTERS.md`), for a second board that would show independent coins exactly and, with other walls, gambler's ruin and first passage. At a 30 mm pitch with 10 mm necks the decisions cannot be told from fair coins; but the bounce moves it (0.3: 69% the same way). A fifth patch tried to take the bounce out (mixing chambers, leaning necks): neither did, and the memory turned out to hang on how a ball meets the neck's mouth (a seam there moved it by 0.13), so no simulation can promise a printed splitter's memory. The plan (a sixth patch): the whole printed tooth is what swaps, in variants of one height so the rows stay put; four sets (necks 10, 12, 14, 20 mm) cover bounces 0.3 to 0.6 within 0.07, tuned on the built board by the pile's spread. A swallowing wall in the test bed (walls.py) gives first passage and the reflection principle as the formulas do with tuned teeth (chi-square 11.6 on 11 df, 3,000 balls), and fails plainly mistuned; two walls give gambler's ruin (three starts, all passing). A gambler's-ruin board wants its walls 4 places apart (2% survive 12 rows; a quarter, a half, three quarters go right). The exhibit (an eighth patch): splitter-board.lua, run by galton.lua through a small BOARD interface (its geometry, its OpenSCAD files, its sign and verdict); every wall mode passes its tour by chi-square. Still to come: its bay in the hall, its placard and lectern card, check-all, its printed parts. Next was: the splitter board in bpp, with swappable wall panels (plain; swallowing, for first passage and the reflection principle; both, for gambler's ruin). Both boards are to be kept. Printed in three tiles under 200 mm, two gears, a stand, an acrylic front sheet; every STL checked watertight, and no two parts overlap at rest. `tools/check-all.sh` (twelve runs, the Galton board's two included) all passed on the revised board. **Not seen by William in the GUI yet.**

- **Done (2026-11-14): the hall re-ordered, the splitter board in it.** William found the old order (by interest alone) off: the Domino Half Adder, which does one thing once, stood sixth; Lehmer's sieve second. MACHINES.md now orders all 48 by what each asks of a visitor (the idea it computes, the mechanism, how much it shows), in five tiers, and the hall walks it: dominoes first, the Galton and splitter boards facing each other in the third pair. The splitter board has its placard, lectern card and check-all lines (fourteen runs).
- **Next (William, 2026-11-14): build the machines from the start of the list on.** The first four (the abacus, the slide rule, Napier's bones, the combination lock) have little or nothing to simulate; the bead and spaghetti sorts (5) are the first with real physics. Each new machine goes in its place in the walking order, the bays after it moving along. The covered bays (the planimeter, the tide predictor, the Differential Analyser, the Difference Engine) wait for their turn in the list. Still offered, not asked for: printing only a Digi-Comp program's parts (the tiles carry every wall; `make-stls.sh` could take a `PARTS` list, with tiles cut anew). Skipped, at William's word: fixing bpp's own `demo/koppi/gears.lua` (its `assign()`, which newer OpenSCAD dropped).
- **Settled (2026-10-21):** William has tried the Pascaline (the mouse and the keys) and it works well; the stylus collision he thought he saw wasn't one. Euler's Gallery keeps its own tables as they are; the exhibit standard is the hall's. The Rec Room's camera flights look good, the top-down table view included.
- **Open, for later:** POV-Ray finds textures only with `+L` the bpp folder (their paths are relative to it): Jakob has it. A printed detent (`detent.stl`, 1.2 mm leaf) would test the stiffness worked out from PLA; William will print one some other time.

8. Opening message for the next chat

> Clone github.com/bullet-physics-playground/bpp, check out the branch test-harness, and read demo/WyomingWill/BabbagesHall/HANDOFF.md, then MACHINES.md and README.md there. Build bpp as described, run tools/check-all.sh, and continue with HANDOFF.md section 7: [the next step, e.g. the planimeter in its covered bay]. Deliver each step as a patch on test-harness, with its documents and PDFs, and the git commands.